# Problems With Monolithic Architecture

# 1. Difficulty in implementing changes

- This is one of the biggest problems
- In a monolithic application, the modules are **tightly coupled/interconnected**
- If we want to make a small change in the **Inventory** module
- we need to **redeploy the entire application** to make that change available



**Why is this a problem ?**

If the application is very large this can result in :-

- Longer **development** cycles
- Longer **deployment** cycles
- More effort for small changes
- Potential impact on unrelated functionality

- **Small change -> potentially full application deployment**

# 2. Lack of Scalability

- Another major problem is that we cannot easily scale **only one specific module.**

Example E-Commerce Application

- Suppose there is a huge Black Friday sale
- The **Inventory Module** is receiving huge traffic because:
  - Users are ordering products
  - Sellers are updating Inventory
  - Inventory is changing rapidly

```
User Module       → Normal traffic
Payment Module    → Normal traffic
Support Module    → Normal traffic
Inventory Module  → VERY HIGH TRAFFIC
```

- Ideally, we would want: **Inventory -> scale this module only**
- But in a monolithic architecture, the **modules are part of same application**
- So we may have to **scale entire application**

**Why is this inefficient ?**

- we are **spending resources** on modules that **don’t actually need** additional capacity
- **Monolithic architecture makes independent scaling difficult and resource-inefficient**

# 3. Long-Term commitment to a Single Technology Stack

- A monolithic application generally commits the organization to a **particular technology stack** for a long period
- Example – **Entire Application – JAVA + Framework X**
- After several years, suppose a newer technology becomes available that would be better for one particular module say Inventory Module.
- Moving the technology may require a **major rewrite/overhaul** of the application
- This can be
  - Expensive
  - Time-consuming
  - Resource-intensive
  - Risky
- **Monolith -> strong log-term commitment to one technology stack**

# 4. Increasing Application Complexity

- A monolith application might start simple

**Problems caused by complexity**

- **Harder** to **understand** the entire application
- **Development** becomes **slower**
- **Deployment** becomes more **difficult**
- More chances of **bugs**
- Changes can have **unexpected effects** on **other components**

For example:

```
Application
├── User
├── Product
└── Order
```

As the company grows:

```
Application
├── User
├── Product
├── Inventory
├── Payment
├── Order
├── Cart
├── Shipping
├── Notification
├── Support
├── Reporting
└── ...
```

The codebase becomes increasingly complex.

# 5. Slower IDE Performance

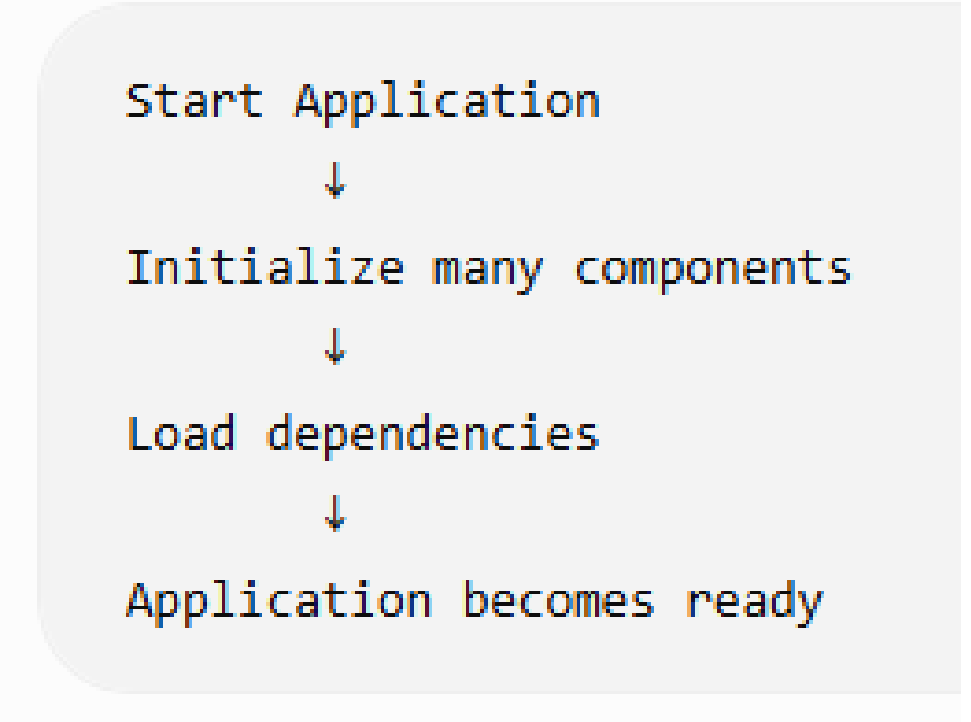
- As the codebase becomes very large, developers may experience problems with their **IDE**.
- Developers may need
  - **More RAM**
  - **More powerful CPU**
  - **Better development machines**
- This can affect
  - IDE **responsiveness**
  - **Build Time**
  - Developer **Productivity**
  - Overall development **speed**

For example:

```
Large Codebase
      ↓
More files
More classes
More dependencies
More components
      ↓
IDE has more work to do
      ↓
Slower development experience
```

# 6. Increased Application Startup Time

- A monolithic application may contain **many components**
- When the application starts, it may need to **initialize**
  - Multiple modules
  - Dependencies
  - Configuration
  - Database-related components
  - Other application components
- **Application startup time can increase.**



# 7. Large Project Size

- This is closely related to the growing **codebase**
- A large monolithic project can become difficult to
  - Manage
  - Understand
  - Maintain
  - Build
  - deploy

```
Small application
      ↓
Easy to understand

Large monolithic application
      ↓
Huge codebase
      ↓
Difficult maintenance
```

# 8. Team Collaboration and Autonomy

- Large organizations often have multiple teams working on different parts of the application.

For an e-commerce application:

```
Team A → Store / User functionality
Team B → Inventory
Team C → Payments
Team D → Orders
Team E → Support
```

All these teams may work within the same large monolithic codebase.

This can create problems with:

**Coordination**

Teams need to coordinate changes.

**Dependencies**

One team's change may depend on another team's code.

**Merge conflicts**

Multiple teams may modify related files/code.

**Delays**

A change may have to wait for another team.

**Reduced autonomy**

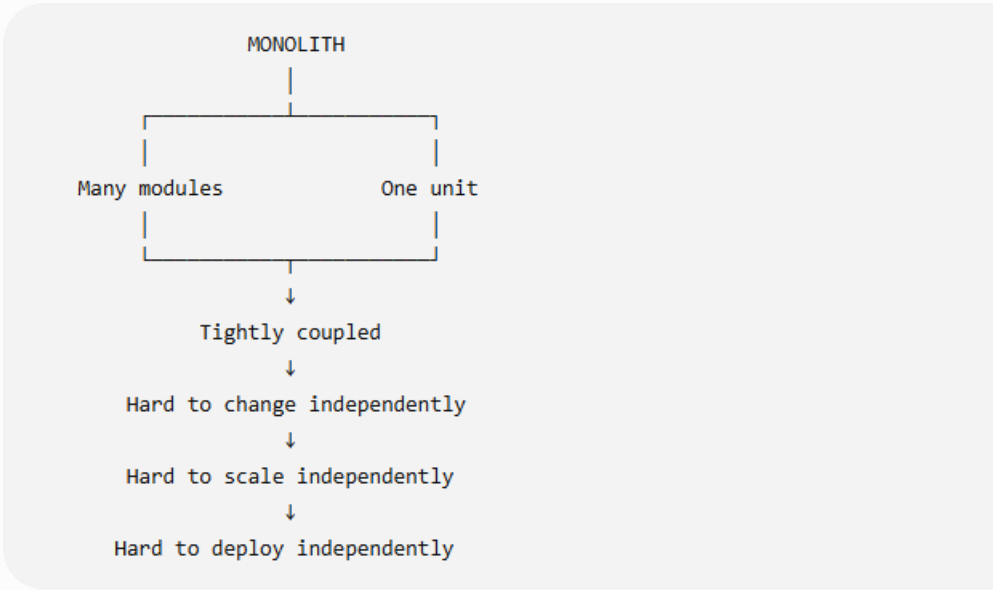
Teams cannot always independently develop, test, release, and deploy their functionality.

So:

> **Large monolithic codebases can make independent team development more difficult.**

# All Problems at a Glance

| Problem | What happens? |
|---|---|
| **1. Difficult changes** | Small changes may require touching/redeploying the whole application |
| **2. Lack of scalability** | Difficult to scale only the heavily used module |
| **3. Technology lock-in** | Difficult/costly to adopt a different technology for one part |
| **4. Increasing complexity** | Large codebase becomes difficult to understand and maintain |
| **5. Slow IDE** | Large codebase can affect local development performance |
| **6. Slow startup** | Many components/dependencies must initialize |
| **7. Large project size** | Management and maintenance become harder |
| **8. Full redeployment** | Small changes require application-level deployment |
| **9. Team collaboration** | Dependencies, conflicts, coordination and delays increase |




This is the key idea you should understand before moving to microservices.